

Simultaneous Reverse Engineering & Feature Identification with Machine Translation

The University of Texas at Dallas
Columbus State University

Hugo Espiritu	hugo.espiritu@utdallas.edu
Hailey Gage	hailey.gage@utdallas.edu
Nick Ramos	nor140030@utdallas.edu
Suleyman Olcay Polat	polat_suleymanolcay@columbusstate.edu

Index terms:

Reverse engineering, decompilation, machine learning , Statistical models

Project Summary

Project Description:

The goal of this project is to modify off-the-shelf machine learning models to decompile binary code with an emphasis on feature identification. We will adapt ideas from recurrent neural net (RNN) decompilation (binary code into C source code) for the seq2seq neural machine translation model (NMT) with an emphasis on feature identification and test the effectiveness on a curated parallel corpus of source and binary code.

Team Responsibilities:

- Create a sufficiently large (50MB+) parallel corpus of source and binary code
- Specify appropriate input and output transformations (pre/post processing, tokenization)
- Specify appropriate evaluation metrics
- Specify appropriate modifications to existing NMT seq2seq model architecture
- Specify features of interest
- Prototype implementation and evaluation on curated parallel corpus

Total Budget:

An estimated ten hours per week will be devoted to this project.

Deliverables:

- Proposal & Literature Review
- Progress Report & Presentation
- Final Report & Presentation
- Documentation & Code
- Dataset

Executive Summary

Title:

Simultaneous Reverse Engineering & Feature Identification with Machine Translation

Affiliation:

University of Texas at Dallas

Columbus State University

Team Members:

Hugo Espiritu

hugo.espiritu@utdallas.edu

Hailey Gage

hailey.gage@utdallas.edu

Nick Ramos

nor140030@utdallas.edu

Suleyman Olcay Polat

polat_suleymanolcay@columbusstate.edu

Index terms:

Reverse engineering, decompilation, machine learning , Statistical models

Summary:

Neural machine translation models are effective at translating between natural languages and can be modified to accommodate stricter structural and grammatical rules of programming languages. This project will adapt ideas from recurrent neural net (RNN) decompilation (binary code into C source code) for the seq2seq neural machine translation model (NMT) with an emphasis on feature identification and test the effectiveness on a curated parallel corpus of source and binary code. For the first part of the project emphasis will be on creating the dataset and checking the accuracy of the model for static data and code separation.

Motivation

Most reverse engineering tools are inefficient, incomplete, and limited in scope. In particular, disassembly listings have a very low abstraction level and require an analyst to manually map assembly instructions to higher level abstractions. The highly repetitive nature of the listings and the need for meticulous attention to changes in small details make the task difficult for beginners and very tedious for experienced analysts. The amount of code analyzed is often massive and may require quick analysis to keep up with adaptive malware and attacks. This domain is an ideal situation for automated analysis with supervised machine learning. The success of neural machine translation (NMT) models for translation of sequences of natural language text inspired

work that modifies NMTs to handle stricter constraints imposed by translation of programming languages. This project will attempt to adapt ideas from recurrent neural net (RNN) decompilation (binary code into C source code) for the seq2seq neural machine translation model (NMT) with an emphasis on feature identification and test the effectiveness on a curated parallel corpus of source and binary code.

Literature Review

Past work in machine learning for decompilation of binary code has focused on a variety of approaches ranging from statistical machine translation to adapting neural machine translation models (NMTs).

The statistical machine translation techniques largely focus on adapting phrase-based statistical machine translation models, use programming language grammars and in some cases also incorporate NLP deep learning models such as Word2Vec [1] [2] [3]. An interesting example is the work by Vasilescu et al [4] that implements a tool that uses off-the-shelf statistical machine translation models (MOSES) to recover some of the original names of Javascript programs minified by the popular tool UglifyJS. The main assumption that is leveraged in this work is the highly repetitive nature of software; this intuition is captured with a statistical language model. This language-independent approach performs well when additional tuning of training data (to account for JS syntax) and some post-processing (to manage scoping) is added. The resulting tool of combining this approach with the previously best performing deobfuscator for Javascript, JSNice (uses static analysis), performs significantly better than either of the individual methods.

The neural machine translation model approaches use large code datasets for byte-equivalent decompilation using RNN encoder-decoder models to learn a decompilation function for translating small snippets of machine code (approximately 100 machine code bytes) to C source code [5]. The main contributions of this work include an NMT architecture modified to translate between programming language representations (including compiled machine code), post processing techniques to improve the RNN decompilation, and useful tokenization schemes to support reverse engineering. However, a variety of challenges remain for making the RNN encoder-decoder architecture a viable replacement to traditional decompilers including support for longer snippets of machine code bytes, ensuring that train sets and test sets don't contain the same code snippets, and verifying that the decompiled code is useful to humans. Another approach that uses modified NMTs to translate between a source language and a target language involves a tree-to-tree encoder-decoder architecture evaluated against several state-of-the-art approaches [6]. The contributions of this work include the ability to handle longer code snippets than sequence-to-sequence models that encode token sequences without structural information and useful metrics for estimating the utility of the decompiled code. The main drawbacks of this approach are the difficulty of generalizing the model to decompile

programs longer than the programs used for training the model and the inability to handle an infinite vocabulary set.

In the specific case of Android applications, Bichsel et al [7] implements a tool, DeGuard, that uses probabilistic graphical models (specifically, conditional random fields) for deobfuscation of Android APKs that are obfuscated with the widely used ProGuard obfuscation tool. This work focuses on recovering identifier names and identifying third party libraries by learning from thousands of readily available, non-obfuscated Android applications and has 79.1\% recovery of program element names, predicts third-party libraries with accuracy of 91.3\% and reports on several examples that suggest that DeGuard can be useful when inspecting malware. The main limitation of DeGuard is the inability to deobfuscate additional data obfuscation techniques that alter data structures, control flow, and common malware obfuscation techniques (eg custom encoding of strings, extensive use of reflection, code reordering using goto statements, etc).

Specific Aims

Our approach will combine these techniques to develop a neural machine translation system that will better handle code snippets of larger size and complexity than existing models on a curated parallel corpus of source and binary code.

Plans

The group will meet weekly to ensure progress and redirect the research if needed. Effective time management and communication will also ensure the project progresses smoothly and on schedule.

Deliverables

- Proposal & Literature Review
- Progress Report & Presentation
- Final Report & Presentation
- Documentation & Code
- Dataset

Midterm Progress

Machine Learning Classifier Selection

We have chosen the following set of machine learning classifiers to try for automated reverse engineering and feature identification:

- Random Forest
- Support Vector Machine
- Recurrent Neural Net (RNN)
- Seq2Seq encoder-decoder framework

Classifier architectures chosen are varied and have different strengths in processing data. Random Forests and Support Vector Machine will give baseline performance to determine whether the sequential architecture of RNN and Seq2Seq classifiers are better suited for classifying the individual bytes of the binaries.

Data Preprocessing and Feature Identification

Current work is focused on the task of automated code-data identification using a selection of machine learning classifiers. We have scripts for extracting the ground truth of function boundaries and function instruction addresses.

Working with the raw bytes (eg input file would be an object dump of the binary where each byte of the binary has its associated class), currently there are seven classes that a byte can be classified as :

"function start",
"function end - instruction",
"function end- not instruction",
"function body - instruction",
"function body - not instruction",
"not function - instruction",
"not function - not instruction"

For example, the following shellcode would classify the first xorl (\x31) as "function start", the last byte (\x80) classified as "function end - not instruction", and every other byte in the shellcode is classified as "function body - instruction" or "function body - not instruction" depending on whether it's an x86 instruction or not.

```
char shellcode[] =
```

```
    "\x31\xc0"        /*  xorl  %eax,%eax  */  
    "\x50"            /*  pushl %eax       */
```

```

"\x68""//sh"      /*  pushl  $0x68732f2f  */
"\x68""/bin"      /*  pushl  $0x6e69622f  */
"\x89\xe3"        /*  movl   %esp,%ebx    */
"\x50"            /*  pushl  %eax         */
"\x53"            /*  pushl  %ebx         */
"\x89\xe1"        /*  movl   %esp,%ecx    */
"\x99"            /*  cdq                      */
"\xb0\x0b"        /*  movb   $0x0b,%al    */
"\xcd\x80"        /*  int    $0x80        */

```

Classifier Evaluation

We are currently in the process of running the classifiers on the pre-processed binaries

Issues

The primary obstacle of this research will be the evaluation metrics of correct behavior and stability of disassembled/decompiled binary code.

Another obstacle will be creating a sufficiently large corpus and ensuring that the train sets and test sets don't contain the same code snippets.

Another issue we may face is uncovering unintended bugs in programs. If a program makes any assumptions about the underlying system, then maintaining correct behavior or stability may not be possible.

Schedule

Sep 28: Submit Proposal

Oct 5: Review NMT project documentation, start corpus creation

Oct 12: Explore different machine learning models for feature identification effectiveness

Oct 19: Try different machine learning models for feature identification,
Start code-data separation task

Prepare presentation and progress report

Oct 26: Submit midterm report

Nov 2: Midterm presentation

Nov 9: Ideation of NMT architecture design, define input/output transformations,
define evaluation metrics, machine learning model performance evaluation

Nov 16: Make modifications on NMT architecture, review work, and pick best performing
machine learning models

Nov 23: Testing

Nov 30: Additional testing and final report draft

Dec 7: Submit final report and presentation

References

- [1] B. E. T. D. P. Allamanis, Miltiadis and C. Sutton, "A survey of machine learning for big code and naturalness," 2017.
- [2] R. V. Karaivanov, Svetoslav and M. Vechev, "Phrase-based statistical translation of programming languages." 2014.
- [3] N. A. T. Nguyen, Trong Duc and T. N. Nguyen, "Mapping api elements for code migration with vector representations," 2016.
- [4] P. D. Bogdan Vasilescu, Casey Casalnuovo, "Recovering clear, natural identifiers from obfuscated js names," 2017.
- [5] E. S. Deborah S. Katz, Jason Ruchti, "Using recurrent neural networks for decompilation," 2018.
- [6] D. S. Xinyun Chen, Chang Liu, "Tree-to-tree neural networks for program translation," 2018.
- [7] P. T. M. V. Benjamin Bichsel, Veselin Raychev, "Statistical deobfuscation of android applications," 2016.

Biographical Sketches of the Investigators

Hugo Espiritu

Hugo Espiritu holds a Bachelor's degree in Mathematics with a double major in Computer Science and is currently pursuing a Master's degree in Computer Science with a specialization in information assurance. Hugo's past experience involves research projects in machine learning and data analytics with cybersecurity applications. Hugo has worked closely with scientists and engineers at government labs (PNNL and US Naval Research lab) on projects such as data visualization of power system models and adversarial machine learning.

Hailey Gage

Hailey Gage holds a Bachelor's degree in Computer Science with a minor in political science and is currently pursuing a Master's degree in Computer Science with a specialization in information assurance. Hailey's past experience involves network security, data application security, digital forensics, public policy, and programming. Hailey has interned at the Internet

Education Foundation as a Technology and Security Policy Intern in the fall of 2017, and interned with Cisco with their Security Solutions Engineer Integration team in the summer of 2016.

Nick Ramos

Nick Ramos holds a Bachelor's degree in Computer Science and is currently pursuing a Master's degree in Computer Science with a specialization in information assurance. Nick's past experience involves network security, web development, and programming. Nick has interned at the General Services Administration with their cyber security team over the summer of 2018. He worked on securing their cloud with Amazon Web Services, assisted with their DevSecOps to provide security tools developed in Java and Python, and helped with incident response.

Suleyman Olcay Polat

Suleyman worked in a computer vision laboratory for 2 years at his university working on creating different machine learning applications for classifying food images and finding their ingredients. After that he started working in the deep learning laboratory for 2 years at his university working on applying deep learning models in Biomedicine and Economics. Their work is published in Neural Computing and Applications. After graduation he started working as a java developer and academic adviser on a simulation project that uses a machine translation method (seq2seq model from google translate) to model and simulate any machine to machine service over internet by listening to their conversations. Currently Suleyman is studying for his Master's degree in computer science and working on creating a chatbot with emotional state implemented within the model. Suleyman has 5 years of experience in the machine learning field and 1 year experience as a professional developer working in a big team.